Sistema de Banco de Dados para Locadora de Veículos: Laboratórios 01 e 02

Arthur Choi Braga
Matrícula: 24101345
Instituto Brasileiro de Ensino,
Desenvolvimento e Pesquisa (IDP)
Brasília, DF, Brasil

Pedro Emanuel da Costa Madruga
Matrícula: 25201037
Instituto Brasileiro de Ensino,
Desenvolvimento e Pesquisa (IDP)
Brasília, DF, Brasil

***Resumo*—Este trabalho consolida os Laboratórios 01 e 02 da disciplina de Banco de Dados, dedicados a um banco de dados relacional para uma locadora de veículos. No Laboratório 01 foram realizados o levantamento de requisitos, o dicionário de dados, a modelagem por meio do Diagrama Entidade-Relacionamento, a normalização até a Terceira Forma Normal e a implementação em SQL, com cinco consultas de negócio. No Laboratório 02 o esquema foi estendido com views, índices avaliados pelo plano de execução (EXPLAIN), triggers de atualização de status e de auditoria, uma stored procedure e uma function. Também foram conduzidos experimentos de concorrência em duas sessões simultâneas: bloqueio explícito com FOR UPDATE, indução e prevenção de deadlock e comparação dos níveis de isolamento READ COMMITTED e REPEATABLE READ sob MVCC. O trabalho é acompanhado de um script SQL final que recria o banco completo.**

***Palavras-chave*—banco de dados relacional, normalização, SQL, índices, triggers, controle de concorrência, MVCC.**

I. Introdução

Uma locadora de veículos precisa controlar quais clientes alugaram quais veículos, em quais filiais as locações começaram e terminaram e se os prazos de devolução foram cumpridos. Registrar essas informações de forma desorganizada gera redundância e inconsistências. Os dois laboratórios desenvolveram um banco de dados relacional para esse cenário, em duas fases.

O Laboratório 01 seguiu as etapas clássicas de projeto [1]: levantamento e dicionário de dados (Seção II), modelagem (Seção III), normalização (Seção IV) e implementação com inserções e consultas (Seção V). O Laboratório 02 partiu desse banco e explorou recursos avançados do SGBD: a evolução do esquema (Seção VI), views e índices (Seção VII), triggers, procedures e functions (Seção VIII) e controle de concorrência (Seção IX). A Seção X descreve o script final consolidado, e as Seções XI e XII trazem a experiência dos integrantes e a conclusão.

Os comandos encontram-se nos arquivos `locadora_final.sql` e `concorrencia_parte3.sql`, entregues na pasta `final` junto com este documento.

II. Etapa 1: Levantamento e Dicionário de Dados

A. Requisitos

Foram identificadas as seguintes necessidades do sistema: (i) cadastrar clientes, identificados unicamente pelo CPF; (ii) organizar os veículos em categorias, cada uma com seu valor de diária; (iii) cadastrar a frota, identificando cada veículo pela placa; (iv) cadastrar as filiais e suas cidades; e (v) registrar as locações, com filial de retirada, filial de devolução (que pode ser diferente), data de retirada, data prevista e data real de devolução. Uma locação cuja data real ainda não foi informada é considerada em aberto.

B. Dicionário de Dados

A Tabela I apresenta o dicionário de dados do Laboratório 01, com o tipo, as restrições e a descrição de cada atributo. As colunas acrescentadas no Laboratório 02 estão na Tabela III.

III. Etapa 2: Modelagem (MER/DER)

A partir do dicionário de dados, foi elaborado o Diagrama Entidade-Relacionamento (DER) apresentado na Fig. 1, construído na ferramenta dbdiagram.io. Os relacionamentos identificados são:

- **Categoria–Veiculo (1:N):** uma categoria agrupa vários veículos, e cada veículo pertence a uma única categoria.
- **Cliente–Locacao (1:N):** um cliente pode realizar várias locações, e cada locação pertence a um cliente.
- **Veiculo–Locacao (1:N):** um veículo pode ser alugado diversas vezes ao longo do tempo.
- **Filial–Locacao (1:N, duas vezes):** cada locação referencia uma filial de retirada e uma filial de devolução, o que é representado por duas chaves estrangeiras distintas para a mesma tabela.

Na implementação física, as chaves primárias foram padronizadas com o nome `id` em todas as tabelas, e as chaves estrangeiras de filial foram nomeadas `id_filial_retirada` e `id_filial_devolucao`, preservando a mesma estrutura do modelo.

Tabela I
DICIONÁRIO DE DADOS

Entidade	Atributo	Tipo de Dado	Restrição	Descrição
Cliente	`id_cliente`	INT	PRIMARY KEY	Identificador único do cliente.
Cliente	`nome`	VARCHAR(100)	NOT NULL	Nome completo do cliente.
Cliente	`cpf`	VARCHAR(14)	UNIQUE, NOT NULL	CPF do cliente cadastrado.
Categoria	`id_categoria`	INT	PRIMARY KEY	Identificador único da categoria.
Categoria	`nome`	VARCHAR(50)	NOT NULL	Nome da categoria.
Categoria	`valor_diaria`	DECIMAL(10,2)	NOT NULL, CHECK > 0	Valor cobrado por dia de locação.
Veiculo	`id_veiculo`	INT	PRIMARY KEY	Identificador único do veículo.
Veiculo	`placa`	VARCHAR(10)	UNIQUE, NOT NULL	Placa de identificação do veículo.
Veiculo	`modelo`	VARCHAR(50)	NOT NULL	Modelo do veículo.
Veiculo	`id_categoria`	INT	FOREIGN KEY	Referência à categoria do veículo.
Filial	`id_filial`	INT	PRIMARY KEY	Identificador único da filial.
Filial	`nome`	VARCHAR(100)	NOT NULL	Nome ou identificação da filial.
Filial	`cidade`	VARCHAR(50)	NOT NULL	Cidade onde a filial está localizada.
Locacao	`id_locacao`	INT	PRIMARY KEY	Identificador único do contrato.
Locacao	`id_cliente`	INT	FOREIGN KEY	Referência ao cliente.
Locacao	`id_veiculo`	INT	FOREIGN KEY	Referência ao veículo alugado.
Locacao	`id_filial_ret`	INT	FOREIGN KEY	Referência à filial de retirada.
Locacao	`id_filial_dev`	INT	FOREIGN KEY	Referência à filial de devolução.
Locacao	`data_retirada`	DATE	NOT NULL	Data em que o cliente retirou o veículo.
Locacao	`data_prevista`	DATE	NOT NULL	Data acordada para a devolução.
Locacao	`data_real`	DATE	NULL	Data efetiva da devolução (vazia se em aberto).

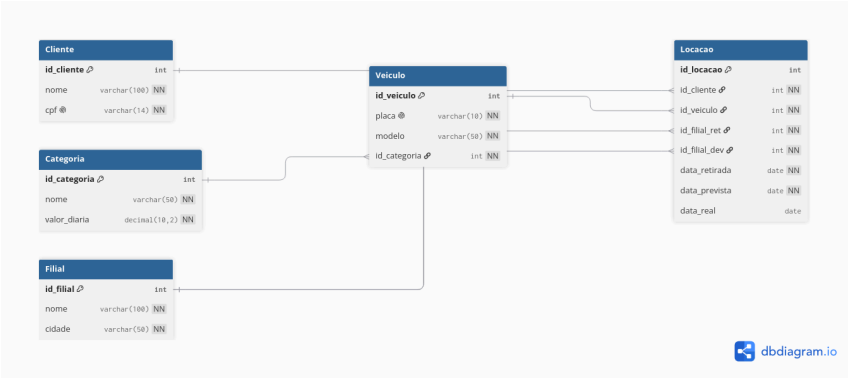


Figura 1. Diagrama Entidade-Relacionamento da locadora de veículos.

IV. ETAPA 3: NORMALIZAÇÃO

A. Análise de Anomalias e Redundâncias

Se todas as informações fossem registradas em uma única tabela geral de locação, os dados do cliente (nome, CPF) e do veículo (placa, modelo, categoria, valor da diária) se repetiriam a cada nova locação. Isso geraria redundância e anomalias de atualização (um reajuste de diária exigiria alterar várias linhas), de inserção (um veículo não poderia ser cadastrado sem uma locação) e de exclusão (apagar a única locação de um cliente eliminaria seu cadastro) [2].

B. Primeira Forma Normal (1FN)

Regra: todos os atributos devem ser atômicos e não podem existir grupos repetidos. *Ação:* os dados foram achatados em colunas únicas para cada registro de locação:

```
Locacao_1FN(id_locacao, id_cliente,
nome_cliente, cpf, id_veiculo, placa, modelo,
nome_categoria, valor_diaria, id_filial_ret,
nome_filial_ret, id_filial_dev, nome_filial_dev,
data_retirada, data_prevista, data_real)
```

Justificativa: o conjunto de datas foi separado em `data_retirada`, `data_prevista` e `data_real`, garantindo que não existam atributos multivalorados.

C. Segunda Forma Normal (2FN)

Regra: a tabela deve estar na 1FN e todos os atributos não-chave devem depender totalmente da chave primária. *Ação:* foram separadas as informações que dependem de outros identificadores que não o da locação:

```
Cliente(id_cliente, nome_cliente, cpf)
Filial(id_filial, nome_filial, cidade)
Veiculo_2FN(id_veiculo, placa, modelo,
```

`nome_categoria, valor_diaria)`
`Locacao(id_locacao, id_cliente, id_veiculo, id_filial_ret, id_filial_dev, data_retirada, data_prevista, data_real)`

Justificativa: se um cliente mudar de nome, a alteração é feita apenas na tabela Cliente, eliminando a redundância.

D. Terceira Forma Normal (3FN)

Regra: a tabela deve estar na 2FN e não pode conter dependências transitivas. *Ação:* na tabela Veiculo, os atributos `nome_categoria` e `valor_diaria` dependem da categoria, e não do veículo (`id_veiculo` → `id_categoria` → `valor_diaria`). Por isso, foi criada a tabela Categoria:

`Categoria(id_categoria, nome_categoria, valor_diaria)`
`Veiculo(id_veiculo, placa, modelo, id_categoria)`

Justificativa: com a categoria isolada, se a locadora aumentar o valor da diária, altera-se apenas uma linha da tabela Categoria. O esquema final, em 3FN, é composto pelas tabelas Cliente, Categoria, Veiculo, Filial e Locacao, correspondendo ao DER da Fig. 1.

V. Etapas 4 e 5: Implementação, Inserções e Consultas

A. Criação das Tabelas

As regras de negócio foram traduzidas em restrições de integridade verificadas pelo próprio SGBD: chaves primárias em todas as tabelas; chaves estrangeiras para as cinco ligações do DER; `UNIQUE` e `NOT NULL` em CPF e placa; e três restrições `CHECK`, que exigem diária positiva (`chk_categoria_valor`), data prevista não anterior à retirada (`chk_datas_previstas`) e data real nula ou não anterior à retirada (`chk_datas_reais`). O banco usa o conjunto de caracteres `utf8mb4`.

B. Inserção de Dados

Foram inseridos cinco registros em cada tabela, respeitando a ordem das dependências de chave estrangeira: Categoria, Filial e Cliente primeiro; em seguida Veiculo; e, por fim, Locacao. A Tabela II resume as locações cadastradas, escolhidas para exercitar os diferentes cenários das consultas: uma locação em aberto, uma devolvida com atraso e três devolvidas no prazo.

Tabela II
Locações Inseridas

id	Cli.	Veíc.	Ret./Dev.	Retirada	Prevista	Real
1	1	1	1 / 2	01/09/26	05/09/26	NULL
2	1	2	2 / 2	10/09/26	12/09/26	14/09/26
3	2	3	1 / 2	20/08/26	20/08/26	20/08/26
4	3	4	3 / 2	25/08/26	25/08/26	25/08/26
5	4	5	4 / 2	02/09/26	02/09/26	02/09/26

C. Consultas

As consultas foram executadas no phpMyAdmin [3] em 16/09/2026. Os resultados estão nas Figs. 2–6.

1) Locações em aberto: seleciona as locações cuja `data_real` é nula, ou seja, veículos ainda não devolvidos. O resultado (Fig. 2) mostra a locação 1, com devolução prevista para 05/09/2026.

2) Veículos disponíveis agrupados: retorna os veículos que não estão em nenhuma locação em aberto (subconsulta com `NOT IN`), agrupados por categoria e modelo. Como o Compass está alugado, restam apenas unidades do Argo, da categoria Econômico (Fig. 3).

3) Cliente com mais locações: conta as locações por cliente (`GROUP BY` e `COUNT`) e ordena de forma decrescente, retornando o primeiro. O cliente 1 possui duas locações (Fig. 4).

4) Faturamento total por filial: multiplica os dias de locação (`DATEDIFF`) pelo valor da diária da categoria e soma por filial de devolução. Para locações em aberto, a data atual (`CURDATE`) é usada no lugar da data real. A filial Aeroporto totalizou R$ 2.570,00 (Fig. 5): 15 dias do SUV a R$ 150,00 da locação 1 somados a 4 dias do Argo a R$ 80,00 da locação 2. Por depender da data atual, esse valor muda conforme o dia de execução.

5) Locações com atraso: seleciona as locações cuja `data_real` é posterior à `data_prevista`. A locação 2 foi devolvida dois dias após o prazo (Fig. 6).

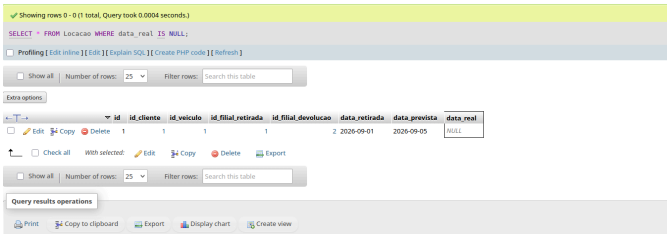


Figura 2. Consulta 1: locações em aberto.

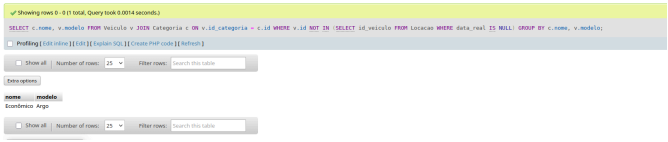


Figura 3. Consulta 2: veículos disponíveis agrupados.

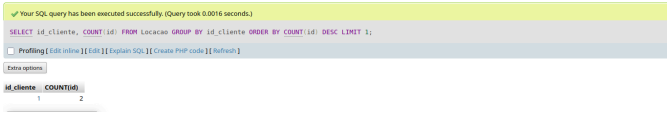


Figura 4. Consulta 3: cliente com mais locações.

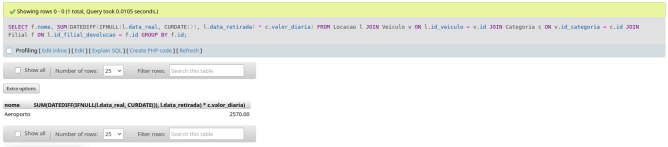


Figura 5. Consulta 4: faturamento total por filial.



Figura 6. Consulta 5: locações com atraso.

VI. Laboratório 02: Evolução do Esquema

O Laboratório 02 reutilizou o banco do Laboratório 01, executado em MySQL 8.4.11 com o mecanismo de armazenamento InnoDB. Para suportar os novos recursos, o esquema recebeu as colunas e a tabela listadas na Tabela III: o estado do veículo, o valor e o estado da locação e uma tabela de auditoria.

Tabela III
Estruturas Acrescentadas no Laboratório 02

Estrutura	Finalidade
`Veiculo.status`	`'disponivel'` ou `'locado'`, mantido por triggers.
`Locacao.valor_total`	Valor calculado na abertura da locação.
`Locacao.status`	Estado do contrato (`'ativa'`, `'finalizada'`).
`log_locacao`	`id_log`, `id_locacao`, valores e status antigo e novo, `usuario` e `data_alteracao`.

VII. Views e Índices

A. Views

Foram criadas três views para simplificar relatórios recorrentes. Uma view armazena apenas a consulta; os dados são lidos das tabelas a cada uso. Assim, quem consulta não precisa conhecer as junções, e o relatório fica padronizado em um único ponto de manutenção.

- **`vw_locacoes_ativas`:** cliente, veículo, placa e datas de retirada e devolução prevista das locações sem data real (junção de Locacao, Cliente e Veiculo).
- **`vw_veiculos_disponiveis`:** veículos com `status = 'disponivel'`, com o nome da categoria e o valor da diária.
- **`vw_faturamento_mensal`:** soma de `valor_total` agrupada por mês/ano (`DATE_FORMAT` sobre a data de retirada) e por filial de retirada. A Tabela IV mostra o resultado sobre os dados do script final, e a Fig. 7 mostra a consulta às três views.

Tabela IV
Resultado de vw_faturamento_mensal

mes_ano	Filial	nome_filial	total_faturado
2026-08	1	Centro	80,00
2026-08	3	Asa Norte	80,00
2026-09	1	Centro	600,00
2026-09	2	Aeroporto	160,00
2026-09	4	Taguatinga	80,00

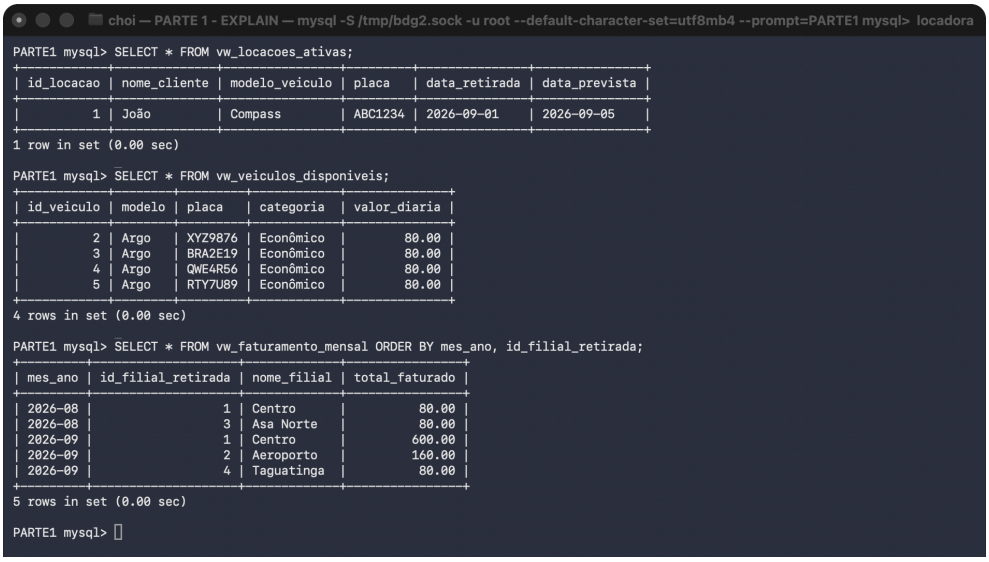


Figura 7. Consulta às três views sobre os dados do script final.

B. Índices e Plano de Execução

Três consultas frequentes foram analisadas com `EXPLAIN` antes e depois da criação dos índices: (A) busca de cliente por CPF; (B) locações por intervalo de data de retirada; e (C) veículos por categoria. Foram criados os índices `idx_locacao_data_retirada`, sobre `Locacao(data_retirada)`, e `idx_veiculo_categoria`, sobre `Veiculo(id_categoria)`. A consulta A já é atendida pelo índice único gerado pela restrição `UNIQUE` do CPF. A Tabela V resume os planos obtidos com o script final, e as Figs. 8 e 9 mostram a saída do `EXPLAIN` antes e depois dos índices.

Tabela V
Plano de Execução Antes e Depois dos Índices

Cons.	Antes: type / key / rows	Depois: type / key / rows
A	`const` / `cpf` / 1	`const` / `cpf` / 1
B	`ALL` / — / 5	`range` / `idx_locacao_data_retirada` / 3
C	`ref` / `id_categoria` / 4	`ref` / `idx_veiculo_categoria` / 4

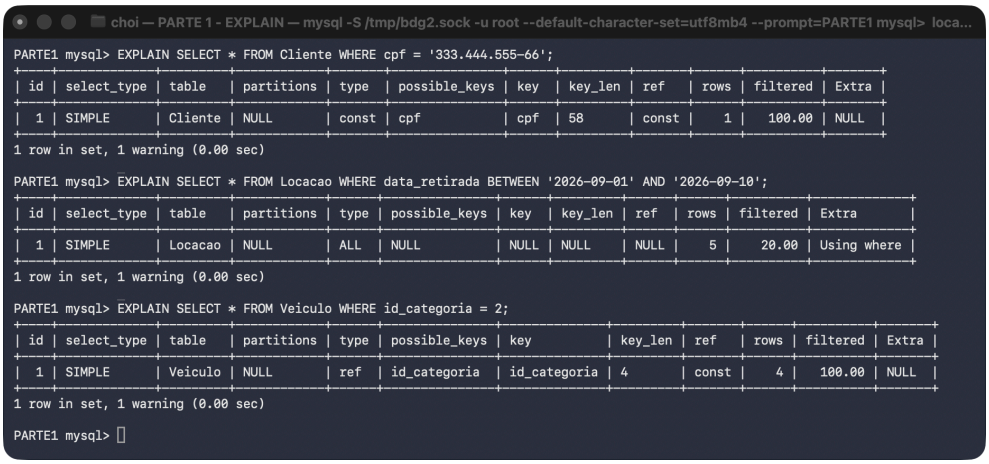


Figura 8. `EXPLAIN` das consultas A, B e C antes da criação dos índices.

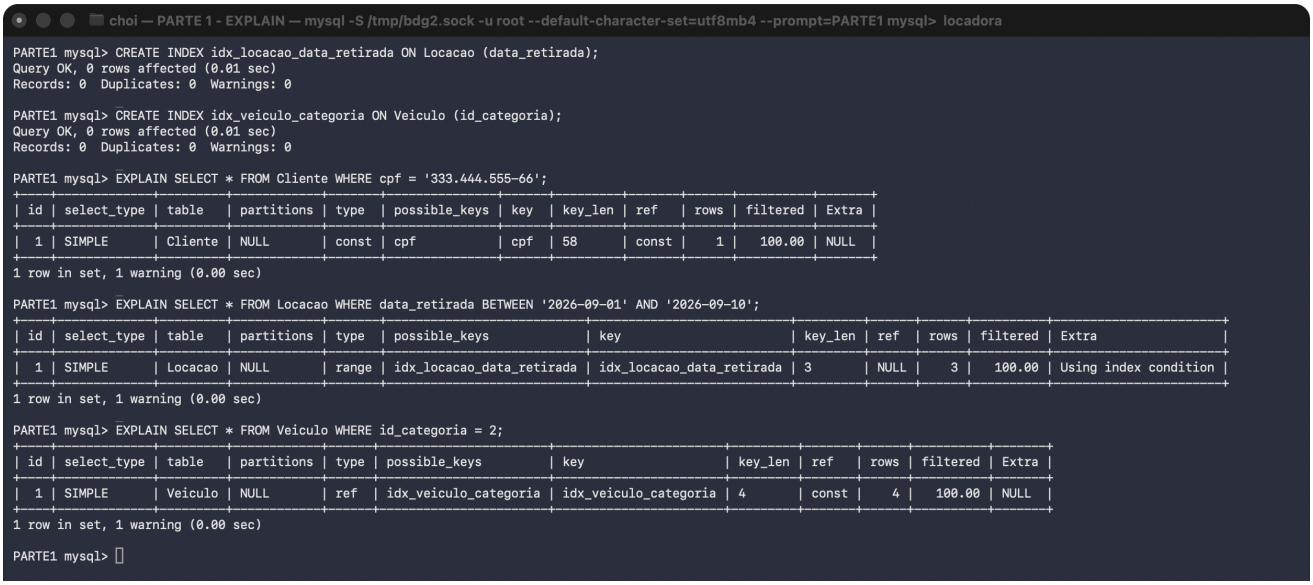


Figura 9. Criação dos índices e `EXPLAIN` das mesmas consultas depois deles.

Na consulta B, o SGBD deixou de varrer a tabela inteira (*Full Table Scan*, tipo `ALL`) e passou a percorrer apenas o intervalo do índice (`range`), com *Using index condition*. O custo estimado (`EXPLAIN FORMAT=JSON`) não diminuiu, passando de 0,75 para 1,61: com apenas cinco linhas, ler a tabela inteira é mais barato que consultar o índice e depois buscar as linhas. O ganho do índice aparece à medida que o volume de locações cresce.

Na consulta C, o plano já usava um índice antes da intervenção: o InnoDB cria automaticamente um índice para cada chave estrangeira, chamado `id_categoria`. Ao criar `idx_veiculo_categoria` sobre a mesma coluna, o MySQL descartou o índice implícito, como mostra `SHOW INDEX`, evitando um índice duplicado. A Fig. 10 registra os custos estimados da consulta B e o `SHOW INDEX` de Veiculo antes e depois.

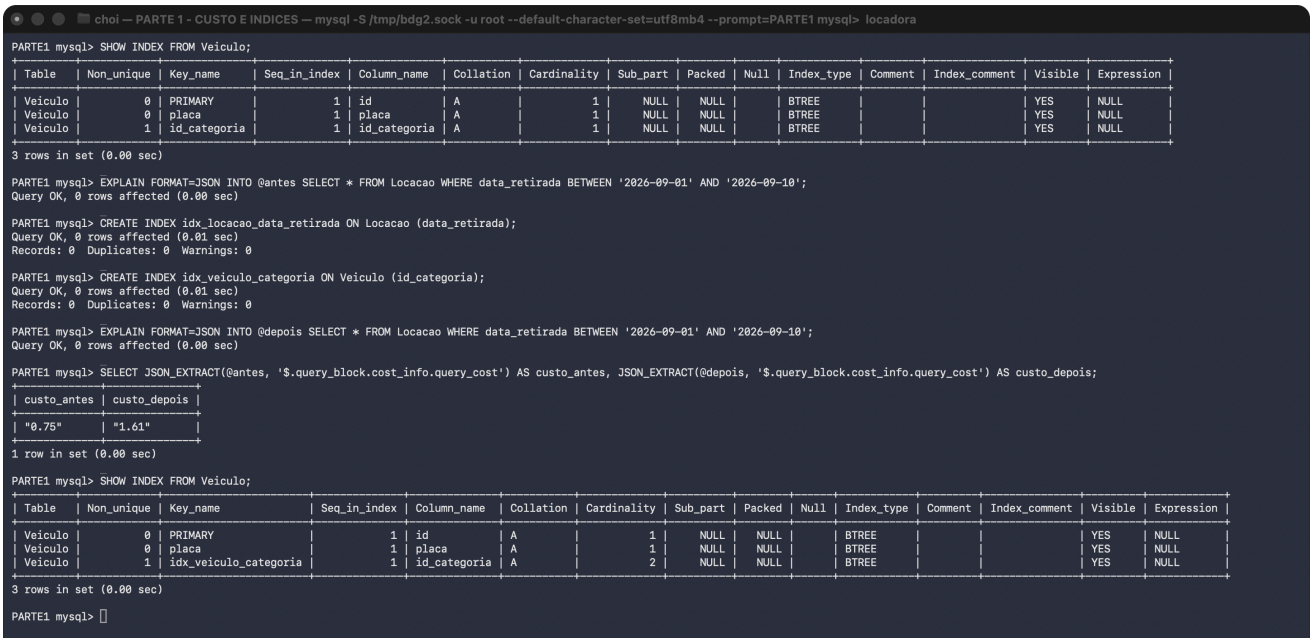


Figura 10. Custo estimado da consulta B (`EXPLAIN FORMAT=JSON`) antes e depois do índice, e índices de Veiculo antes e depois de `idx_veiculo_categoria`.

C. Reflexão: Excesso de Índices

Índices aceleram leituras, mas têm custo. Cada `INSERT`, `UPDATE` ou `DELETE` precisa atualizar todos os índices da tabela, o que torna as escritas mais lentas e aumenta a contenção de locks. Os índices também ocupam disco e disputam o *buffer pool* com os dados, e um grande número de alternativas pode levar o otimizador a escolher planos piores. Em tabelas com alta taxa de escrita, como Locacao em um período de muitas aberturas e devoluções, convém manter apenas os índices que atendem consultas realmente frequentes e seletivas.

VIII. Triggers, Procedures e Functions

A. Triggers

Foram criados três triggers do tipo `AFTER` sobre a tabela Locacao, que transferem para o banco regras que, de outra forma, dependeriam da aplicação:

- **`trg_locacao_insert`:** ao inserir uma locação, altera o status do veículo para `'locado'`.
- **`trg_locacao_update`:** quando `data_real` passa de nula para preenchida, devolve o veículo ao status `'disponivel'`.
- **`trg_auditoria_locacao`:** se `valor_total` ou `status` da locação mudar, insere em `log_locacao` os valores antigo e novo, o usuário (`CURRENT_USER()`) e o instante (`NOW()`).

Listagem 1. Trigger de auditoria.

```
CREATE TRIGGER trg_auditoria_locacao
AFTER UPDATE ON Locacao FOR EACH ROW
BEGIN
  IF (OLD.valor_total <> NEW.valor_total)
     OR (OLD.status <> NEW.status) THEN
    INSERT INTO log_locacao (id_locacao,
      valor_antigo, valor_novo, status_antigo,
      status_novo, usuario, data_alteracao)
    VALUES (OLD.id, OLD.valor_total,
      NEW.valor_total, OLD.status, NEW.status,
      CURRENT_USER(), NOW());
  END IF;
END
```

B. Procedure `sp_abrir_locacao`

A procedure recebe cliente, veículo, filial e quantidade de dias e executa três passos: (1) consulta o status do veículo e, se não estiver disponível, interrompe a execução com `SIGNAL SQLSTATE '45000'`; (2) busca o valor da diária da categoria; e (3) calcula o total (diária × dias) e insere a locação com data prevista igual à data atual somada aos dias. Ao inserir, a procedure dispara `trg_locacao_insert`, que marca o veículo como locado.

C. Function `fn_calcula_multa`

A function recebe o identificador da locação e a taxa diária de multa. Se a devolução ocorreu depois da data prevista, retorna os dias de atraso (`DATEDIFF` entre data real e data prevista) multiplicados pela taxa; caso contrário, retorna zero.

Listagem 2. Corpo da function de multa.

```
SELECT data_prevista, data_real
  INTO v_prevista, v_real
  FROM Locacao WHERE id = p_id_locacao;
IF v_real IS NOT NULL AND v_real > v_prevista THEN
  SET v_dias_atraso = DATEDIFF(v_real, v_prevista);
  RETURN v_dias_atraso * p_taxa_diaria_multa;
END IF;
RETURN 0.00;
```

D. Testes

Os objetos foram testados no script final dentro de uma transação desfeita ao término (`ROLLBACK`), de modo que o banco permanece com os dados de exemplo. Os resultados estão na Tabela VI e nas Figs. 11 e 12.

Tabela VI
TESTES DA PARTE 2

Teste	Resultado
`fn_calcula_multa(2, 50.00)`	100,00 (2 dias de atraso)
`fn_calcula_multa(3, 50.00)`	0,00 (devolvida no prazo)
`CALL sp_abrir_locacao(5, 3, 1, 3)`	Locação 6 criada, valor R$ 240,00; veículo 3 passa a `'locado'`
`CALL sp_abrir_locacao(2, 1, 1, 2)`	`ERROR 1644 (45000)`: veículo indisponível
Devolução da locação 1 (`UPDATE` de `data_real` e `status`)	Veículo 1 volta a `'disponivel'`; uma linha em `log_locacao` (`ativa` → `finalizada`)

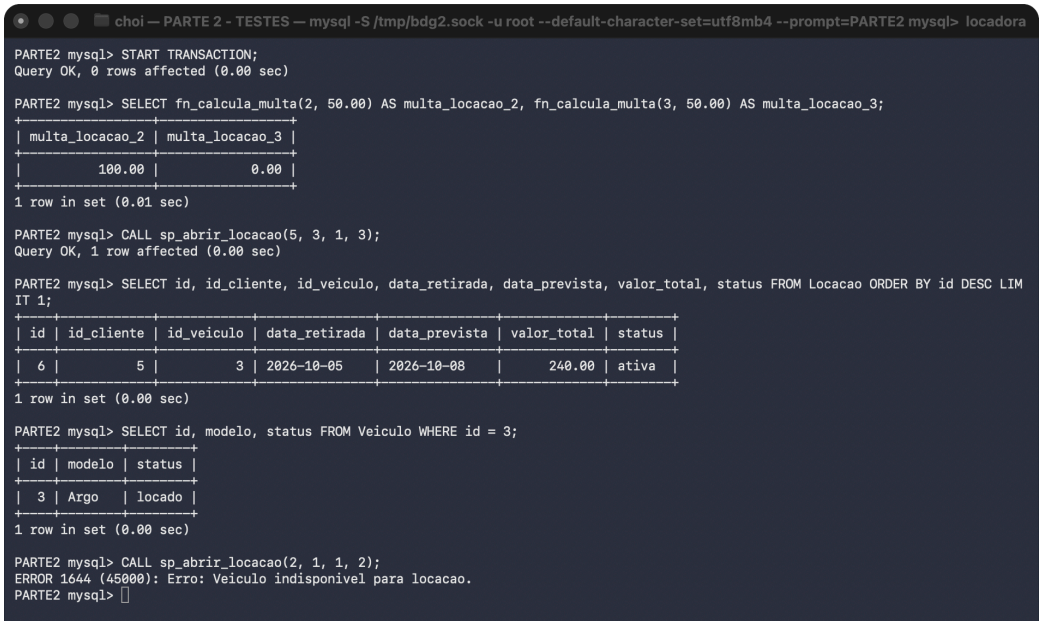


Figura 11. Testes de `fn_calcula_multa` e `sp_abrir_locacao`, incluindo a rejeição de um veículo indisponível.

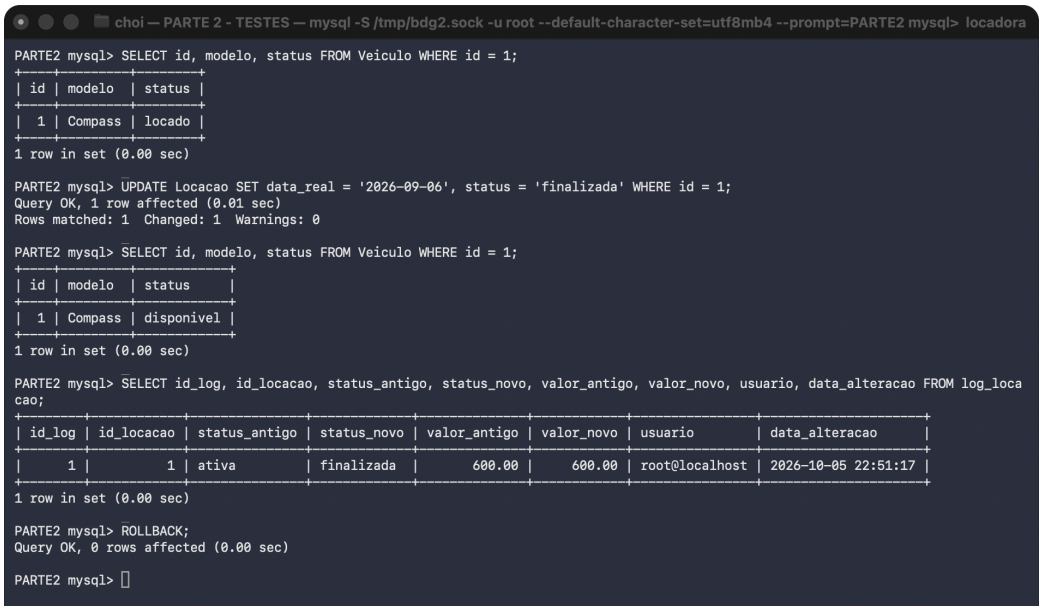


Figura 12. Devolução da locação 1: status do veículo antes e depois, registro em `log_locacao` e `ROLLBACK`.

IX. Concorrência e Transações

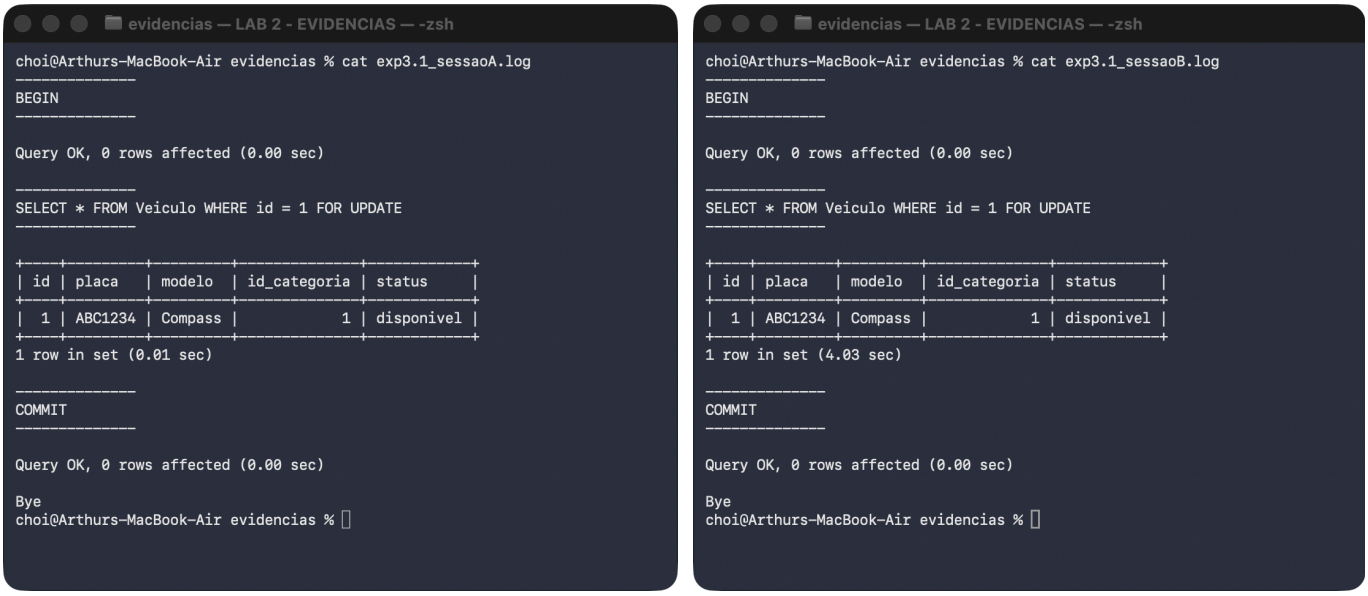
Os experimentos foram realizados em 28/09/2026 em duas janelas independentes do terminal, cada uma com sua própria conexão ao MySQL: Sessão A, operada por Pedro, e Sessão B, operada por Arthur. Cada janela recebeu do servidor um *connection id* distinto, comprovando que as sessões eram concorrentes. Uma terceira sessão foi usada apenas para inspecionar o dicionário de bloqueios. Todos os experimentos terminaram com `ROLLBACK` ou com a restauração explícita dos valores.

A. Experimento 3.1: Bloqueio Explícito (`FOR UPDATE`)

Simulou-se a tentativa de dois atendentes reservarem o mesmo veículo. A Sessão A abriu uma transação e executou `SELECT * FROM Veiculo WHERE id = 1 FOR UPDATE` sem confirmar. A Sessão B executou o mesmo comando e ficou bloqueada. A consulta da terceira sessão a `information_schema.innodb_trx` e `performance_schema.data_lock_waits` mostrou a *thread* 17 (Sessão B) no estado `LOCK WAIT`, aguardando um recurso retido pela *thread* 16 (Sessão A). Após o `COMMIT` de A, a Sessão B foi liberada imediatamente. A Tabela VII traz os tempos registrados nas transcrições das sessões (Fig. 13).

Tabela VII
Tempos de Resposta do Experimento 3.1

Sessão	Comando	Tempo
A	`SELECT ... FOR UPDATE`	0,01 s
B	`SELECT ... FOR UPDATE` (idêntico)	4,03 s



(a) `exp3.1_sessaoA.log` (b) `exp3.1_sessaoB.log`

Figura 13. Transcrições das sessões no Experimento 3.1.

O `SELECT ... FOR UPDATE` adquire um lock exclusivo (X) sobre a linha lida. Enquanto a transação detentora não termina, outra transação que tente obter o mesmo lock é serializada: espera em vez de ler um dado prestes a mudar. É esse mecanismo que impede que dois atendentes reservem o mesmo veículo simultaneamente.

B. Experimento 3.2: Indução de Deadlock

A Sessão A atualizou o cliente 1 (lock em Cliente); a Sessão B atualizou o veículo 1 (lock em Veiculo). Em seguida, A tentou atualizar o veículo 1 e ficou aguardando B; por fim, B tentou atualizar o cliente 1, fechando o ciclo. O MySQL retornou à Sessão B a mensagem da Fig. 14:

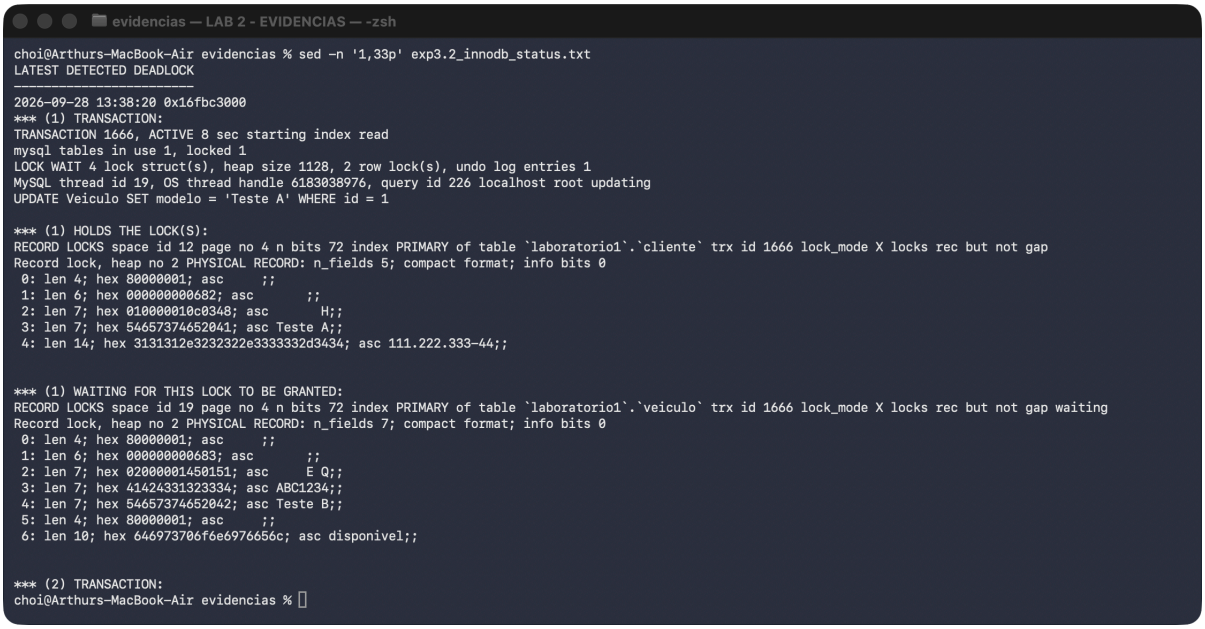
```
ERROR 1213 (40001): Deadlock found when trying
to get lock; try restarting transaction
```



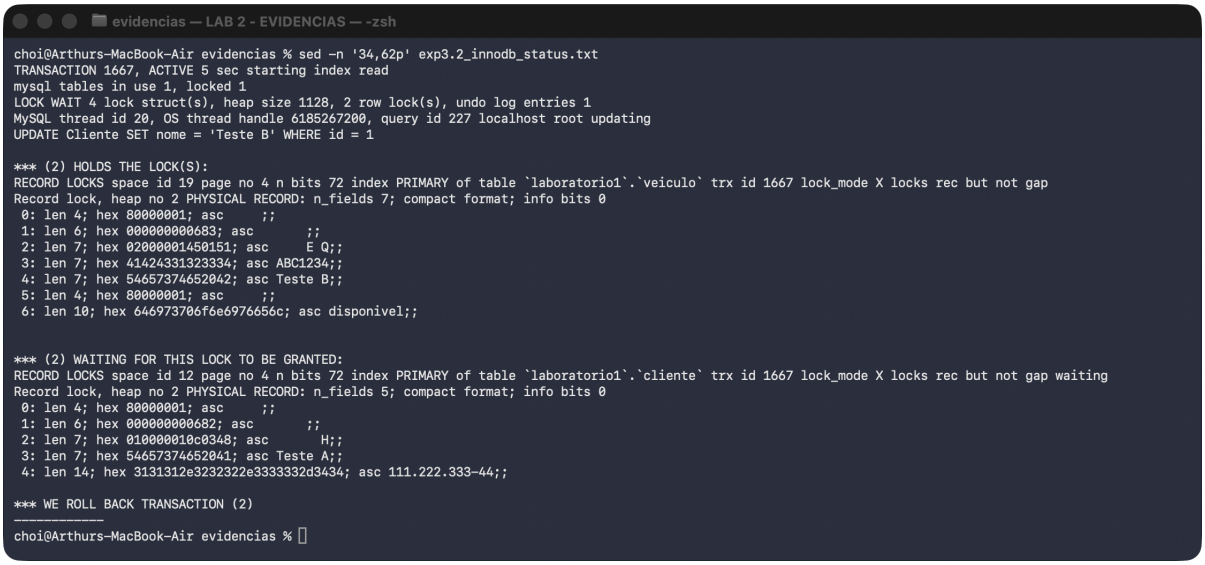
Figura 14. Sessões A (esquerda) e B (direita) no momento do deadlock.

O relatório de `SHOW ENGINE INNODB STATUS` expôs o ciclo de espera: a transação 1666 (Sessão A) detinha o lock de Cliente e esperava o de Veiculo, enquanto a transação 1667 (Sessão B) detinha o lock de Veiculo e esperava o de Cliente. O InnoDB detectou o ciclo em seu grafo de espera (*wait-for graph*), escolheu como vítima a transação de menor custo de desfazimento e a reverteu (`WE ROLL BACK TRANSACTION (2)`). No mesmo instante, a Sessão A concluiu seu `UPDATE`

em 4,27 s. A Fig. 15 mostra o relatório do detector de deadlock.



(a) Transação 1666 (Sessão A): locks detidos e aguardados



(b) Transação 1667 (Sessão B) e a decisão `WE ROLL BACK TRANSACTION (2)`

Figura 15. Seção `LATEST DETECTED DEADLOCK` do `SHOW ENGINE INNODB STATUS` (arquivo `exp3.2_innodb_status.txt`).

C. Prevenção de Deadlocks

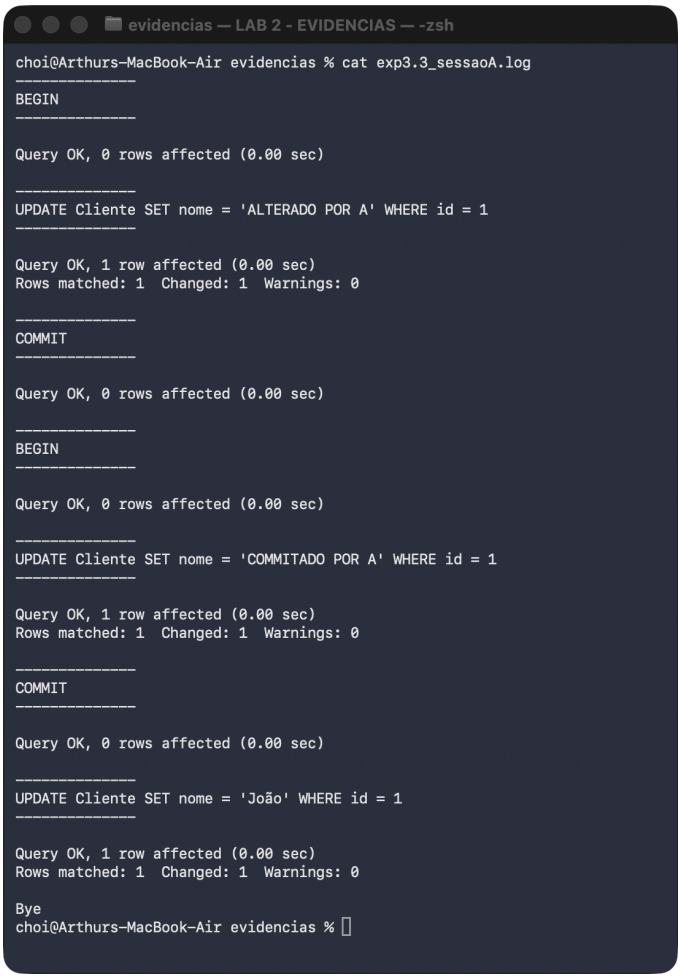
O deadlock não foi causado pelos dados, e sim pela ordem divergente de acesso: A bloqueou Cliente e depois Veiculo, enquanto B fez o inverso. A prevenção consiste em adotar, em toda a aplicação, uma ordem global e única de acesso às tabelas, por exemplo a alfabética (Categoria → Cliente → Filial → Locacao → Veiculo). Se ambas as sessões bloqueassem Cliente antes de Veiculo, a segunda apenas esperaria no primeiro recurso, sem possuir nada de que a primeira precisasse. Sem posse cruzada não há espera circular, uma das quatro condições necessárias para o deadlock [4]; o resultado passa a ser um simples enfileiramento, como no Experimento 3.1. Medidas complementares são manter as transações curtas, atualizar linhas pela chave primária (travando apenas uma linha) e reexecutar automaticamente a transação ao receber `SQLSTATE 40001`, como a própria mensagem do SGBD sugere.

D. Experimento 3.3: Níveis de Isolamento e MVCC

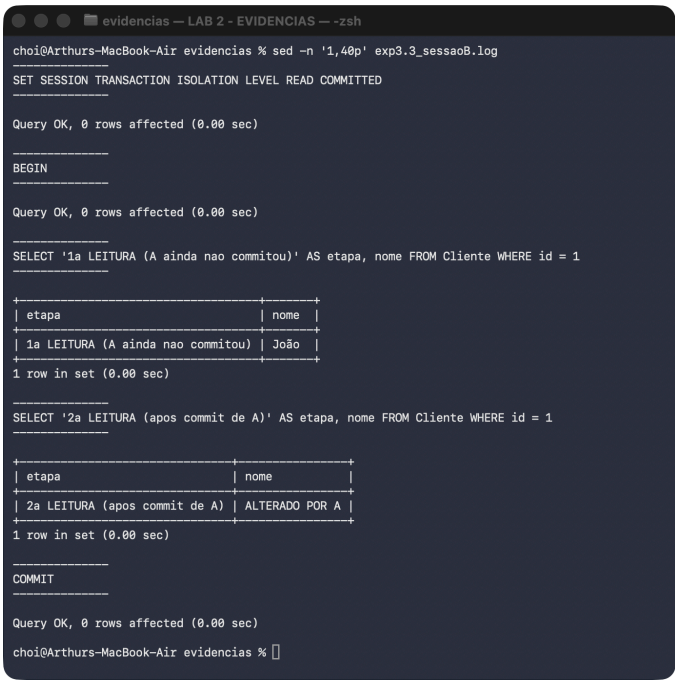
No primeiro teste, a Sessão B usou `READ COMMITTED`. A Sessão A alterou o nome do cliente 1 sem confirmar, e B continuou lendo o valor antigo, reconstruído a partir do *undo log*: não houve leitura suja. Após o `COMMIT` de A, a mesma consulta, na mesma transação de B, retornou o valor novo, caracterizando a leitura não repetível. No segundo teste, com `REPEATABLE READ`, B criou o snapshot na primeira leitura e continuou enxergando o mesmo valor mesmo depois de A alterar e confirmar o registro; o valor novo só apareceu após o término da transação de B (Tabela VIII e Figs. 16 e 17).

Tabela VIII
LEITURAS DA SESSÃO B NO EXPERIMENTO 3.3

Nível	Antes do COMMIT de A	Depois do COMMIT de A
READ COMMITTED	João	ALTERADO POR A
REPEATABLE READ	ALTERADO POR A	ALTERADO POR A (mantido)



(a) `exp3.3_sessaoA.log`



(b) `exp3.3_sessaoB.log`, linhas 1–40 (`READ COMMITTED`)

Figura 16. Transcrições do Experimento 3.3: Sessão A e teste da Sessão B com `READ COMMITTED`.

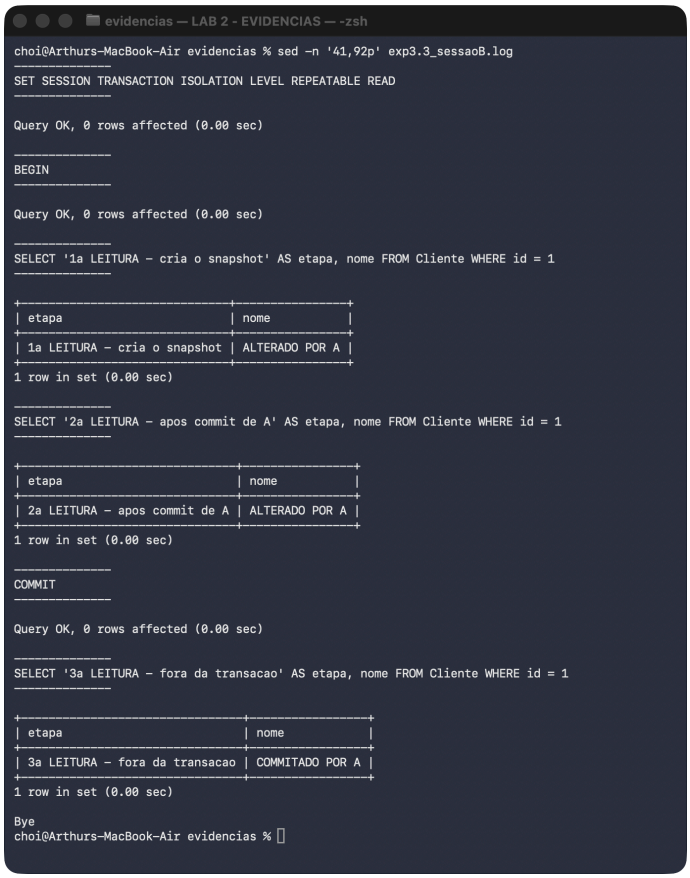


Figura 17. `exp3.3_sessaoB.log`, linhas 41–92: teste da Sessão B com `REPEATABLE READ`.

O MVCC (*Multi-Version Concurrency Control*) mantém várias versões de um mesmo registro, de modo que leituras não bloqueiam escritas e escritas não bloqueiam leituras: em nenhum dos testes a Sessão B precisou esperar. A diferença entre os níveis está no momento em que o snapshot é tirado: a cada comando em `READ COMMITTED` e na primeira leitura da transação em `REPEATABLE READ`, que é o nível padrão do InnoDB. Para um relatório financeiro, que executa várias consultas relacionadas e precisa de um retrato consistente, `REPEATABLE READ` é o nível adequado; para uma consulta pontual de disponibilidade, `READ COMMITTED` é suficiente.

X. Script Final Consolidado

Os laboratórios produziram scripts separados: `locadora.sql` (Laboratório 01), a exportação do phpMyAdmin com os objetos do Laboratório 02 e o roteiro da Parte 3. O arquivo `locadora_final.sql` une tudo em um script que recria o banco `locadora` do zero, na ordem: estrutura, dados, consultas, views, índices com `EXPLAIN` antes e depois, triggers, procedure, function e testes. O arquivo `concorrencia_parte3.sql` mantém os comandos dos experimentos de concorrência, que exigem duas sessões. Na consolidação foram feitos os seguintes ajustes:

- As restrições do Laboratório 01 (`NOT NULL`, `CHECK` e a coluna `Filial.cidade`), ausentes na exportação do Laboratório 02, foram mantidas.
- `Locacao.id` passou a ser `AUTO_INCREMENT`, pois `sp_abrir_locacao` insere locações sem informar o identificador.
- Os dados de exemplo receberam status e `valor_total` coerentes: o veículo 1 começa locado, as locações devolvidas ficam finalizadas e o valor é diária × dias previstos, com mínimo de uma diária.
- Os triggers são criados após a carga inicial, para não alterarem o status dos dados de exemplo.
- `fn_calcula_multa` foi declarada `READS SQL DATA` em vez de `DETERMINISTIC`, pois seu resultado depende do conteúdo da tabela.
- Foram acrescentados `CHECK` para o status do veículo e para valor total não negativo.

O script foi executado sem erros em 05/10/2026, em MySQL 8.4.11; as Tabelas IV, V e VI e as Figs. 7 a 12 reproduzem as saídas dessa execução.

XI. Experiência dos Integrantes

Arthur Choi Braga (24101345): no Laboratório 01, ficou responsável pelos scripts SQL, escrevendo os comandos de criação das tabelas, a adaptação das inserções para cinco registros por tabela e as consultas. Também organizou o documento final no formato IEEE e o repositório de entrega no GitHub. No Laboratório 02, operou a Sessão B nos experimentos de concorrência e documentou as evidências da Parte 3 (transcrições das sessões, print do deadlock e relatório do `SHOW ENGINE INNODB STATUS`), além de organizar o repositório de entrega.

Pedro Emanuel da Costa Madruga (25201037): no Laboratório 01, realizou o levantamento de requisitos e o dicionário de dados, elaborou o DER no dbdiagram.io e conduziu

a normalização até a 3FN. Também executou as consultas no phpMyAdmin e registrou os prints dos resultados. No Laboratório 02, criou as views e os índices e conduziu a análise com `EXPLAIN`, incluindo a reflexão sobre o excesso de índices; adaptou o esquema e implementou os triggers, a procedure `sp_abrir_locacao` e a function `fn_calcula_multa`; e operou a Sessão A nos experimentos de concorrência.

XII. Conclusão

Os laboratórios percorreram o ciclo de vida de um banco de dados relacional. A normalização até a 3FN eliminou as redundâncias da tabela única inicial, e as restrições de integridade garantem a consistência dos dados. As views simplificaram os relatórios, e a análise com `EXPLAIN` mostrou tanto o ganho dos índices quanto seus limites em tabelas pequenas. Triggers, procedure e function levaram regras de negócio para dentro do SGBD, e os experimentos de concorrência evidenciaram como locks, detecção de deadlock e MVCC preservam a coerência dos dados com vários usuários simultâneos.

Como trabalho futuro, a verificação de disponibilidade em `sp_abrir_locacao` pode usar `SELECT ... FOR UPDATE`, como no Experimento 3.1. Hoje a consulta de status não trava a linha, e duas chamadas simultâneas para o mesmo veículo poderiam passar juntas pela validação.

Referências

[1] R. Elmasri and S. B. Navathe, *Sistemas de Banco de Dados*, 7th ed. São Paulo, Brasil: Pearson, 2018.

[2] C. J. Date, *Introdução a Sistemas de Bancos de Dados*, 8th ed. Rio de Janeiro, Brasil: Elsevier, 2004.

[3] Oracle Corporation, “MySQL 8.4 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/

[4] E. G. Coffman, M. Elphick, and A. Shoshani, “System deadlocks,” *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, 1971.